

Speaker Notes — NetSquid for Quantum Networks

CQT Singapore, 29 Jul 2026

Slide 1 — Why Simulate Quantum Networks?

The motivation is the gap between component specs and live network behaviour. Analytical models (Shor-Preskill, PLOB bound) give clean closed-form results but require a new derivation for every noise combination. NetSquid lets you compose arbitrary noise and protocol logic and read out metrics immediately — the reconfigurability is the point.

The "?" box in the diagram is deliberate: simulation fills that gap, but it introduces its own limits (the rest of the talk).

Likely question: "Why not just use analytical bounds?" Answer: bounds like PLOB tell you the ceiling, not what a specific protocol with specific hardware achieves. Simulation gives you the achievable rate under realistic imperfections, not just the limit.

Slide 2 — NetSquid: Discrete-Event Simulator

NetSquid is a TNO/TU Delft tool, free for research use. It sits on PyDynAA — a bespoke Cython discrete-event layer. Three core abstractions:

- `Component` : physical device (source, detector, fibre channel)
- `Node` : network node that owns components
- `Protocol` : event-driven logic that runs on nodes

Everything is composable. You add a noise model by swapping a model object, not rewriting protocol logic.

Key point for CQT audience: This is not a circuit simulator. It is a network simulator that happens to simulate quantum channels — closer to ns-3 or OMNeT++ than Qiskit.

Slide 3 — Bridging Device Physics to Network Modelling

The workflow: measured device parameters go in, network-level metrics (key rate, QBER, pair success rate) come out. NetSquid sits in the middle composing noise and executing protocol logic.

"What you provide / what you gain" — this is the practical pitch. You don't need to rederive anything; you change a parameter in a JSON config and re-run.

Slide 4 — Running Example: BB84 and MDI-QKD

BB84: prepare-and-measure. Alice sends photons, Bob measures. Bob's detector is trusted — if an adversary has access to Bob's detector, the protocol breaks. Key rate scales linearly with detector efficiency: $R \propto \eta_d$.

MDI-QKD (Lo, Curty & Qi, 2012): both Alice and Bob send to an untrusted relay Charlie who performs a Bell state measurement (BSM). Neither detector is trusted. Rate scales as $R \propto \eta_d^2$ because coincidence requires both photons detected — quadratic penalty. This is the fundamental MDI trade-off: detector-side attack immunity at the cost of lower rate.

At network scale: BB84 needs $O(N^2)$ direct fibre links between all user pairs. MDI only needs $O(N)$ links (each user to their relay) — much cheaper infrastructure.

TN-BB84 (used in network simulations): relay runs BB84 with each user, then XORs the keys. Relay holds key material — must be trusted. Purely classical at the relay (no entanglement swapping). This is standard in the literature for trusted-node networks (e.g., the Tokyo QKD network).

Slide 5 — Methodology: Layered Physical Modelling

Ten layers, each adding one physical imperfection on top of the previous. Layer 0 is ideal; layer 9 is fully physical for MDI.

Key point: **each layer is a model swap, not a protocol rewrite.** NetSquid hooks — Beer-Lambert loss, DephaseNoiseModel, BSM success probability — are independent of the protocol logic above them. This is the architectural advantage of NetSquid for this kind of study.

MDI-specific layers (8, 9) add beam splitter efficiency and asymmetric relay placement. Charlie placement peaks at midpoint — symmetric arms equalise per-photon loss.

Slide 6 — P2P Parameter Sweeps

Four headline findings:

1. **Distance:** both protocols show same exponential decay — total fibre loss identical for same total length. Greater total insertion loss as well as detector and beam-splitter

inefficiencies causes the MDI offset below BB84.

2. **Detector efficiency η_d :** BB84 linear, MDI quadratic. The quadratic drop emerges naturally from simulation — you don't have to know this in advance, NetSquid produces it.
3. **Node/connector loss:** MDI steeper because both arms lose independently — effectively $(1-p)^2$ vs $(1-p)$.
4. **Charlie placement:** peaks at midpoint. Symmetric arms is optimal.

Why this matters for the talk: "NetSquid makes the η_d^2 dependence emerge from simulation." You didn't hard-code it; it came from the coincidence detection model. That's the demonstration of the tool's power.

Slide 7 — Network Methodologies

MDI/TN-BB84 (left in topology viz): star-mesh. Each user connects to nearest relay (star); relays connect to each other (mesh backbone). Cross-cluster pairs go through two user-relay links and one backbone link.

BB84 full mesh (right in topology viz): every user pair has a direct fibre link. $O(N^2)$ links — at $N=13$ this is 78 links vs $13+3$ for MDI.

Relay placement: k-means clustering. Optimality of centroid placement confirmed by `relay_placement.py` experiment 1 (single cluster sweep — yellow peak converges to red star centroid across all user counts and seeds).

TN-BB84 relay: XOR chain. Relay computes $K_{AB} = K_{AR} \oplus K_{BR}$ where K_{AR} and K_{BR} are the keys from each user-relay BB84 session. No quantum operation at relay — purely classical. Backbone key diluted by number of cross-relay pairs sharing it.

Slide 8 — Network Scale Results

Two sweeps: relay count K and user count N .

Rate vs K : flat for both protocols once $K \geq 2$. Adding more relays doesn't improve key rate — the relay is not the bottleneck, the user-relay link quality is. This is a non-obvious result: you might expect more relays = better performance.

Rate vs N — BB84: flat. Direct links mean path length = user-user distance. In a fixed area, average inter-user distance is independent of N (expected distance between two uniform random points in a square depends only on area, not point count). Rate stays constant.

Rate vs N — MDI: declining. MDI path = $L_{AC} + L_{BC}$ (user $\rightarrow$ relay + user $\rightarrow$ relay, two independent quantum channels). By the triangle inequality, $L_{AC} + L_{BC} \geq |AB|$, with

equality only when the relay lies exactly on the line segment AB — almost never true for random placement. As N grows with fixed K relays, more cross-cluster pairs appear, and most sit on the "wrong side" of their relay (the relay is the centroid of its own cluster, optimised for within-cluster geometry, not for any cross-cluster pairing). The detour penalty accumulates: database confirms `avg_pair_distance_km` rises from ~18.1 km at $N=11$ to ~19.0 km at $N=19$ for MDI, while BB84 stays flat at ~12.7 km.

Rate gap: MDI sits ~10× below BB84 throughout — η_d^2 baseline penalty (quadratic vs linear detector efficiency dependence) compounded by the relay detour loss on cross-cluster pairs.

Slide 9 — Bottleneck: Computational Cost

Three root causes composing:

1. **Sequential event loop:** within each MC run, events are causally ordered and cannot be parallelised. This is intrinsic to DES, not an NetSquid implementation choice. Bel et al. (2024) confirm this is the generic bottleneck.
2. **$O(N^2)$ pair count:** for N users, you simulate $N(N-1)/2$ user pairs. At $N=100$: 4950 pairs. Each pair needs its own set of MC runs.
3. **Spawn overhead:** each Monte Carlo batch launches a new OS process (~2-3s regardless of workload). At 10 batches of 100 runs, that's 20-30s overhead on top of simulation time.

Result: P2P sweep ~minutes. $N=20$ relay sweep ~hours. $N=100$ sweep ~weeks (infeasible in practice).

Quantum state processing is NetSquid's largest generic bottleneck per Coopmans et al. (2021) — but for QKD specifically it's negligible because you never process more than 2 qubits at once (BB84: 1 qubit; MDI: 2-qubit BSM). The bottleneck here is architectural, not quantum-mechanical.

More computationally expensive applications of quantum networking (distillation, QML, large-state teleportation) are limited significantly by quantum processing steps. These might require state compression in addition to architectural changes.

Likely question — "Could you map CPU cores to network nodes and parallelize that way?" Yes — this is the process-per-node variant of PDES, and it works up to a point. Assign one core per node; inter-core communication models inter-node traffic; independent nodes process events genuinely in parallel. A university HPC cluster (~500–2000 cores, e.g. UCL Myriad ~50k cores across ~800 nodes) could in principle simulate a 500-node quantum network this way — far beyond what NetSquid handles serially. However quantum non-locality puts a synchronisation tax on exactly the operations you want to parallelise: when Alice and Charlie share a Bell pair, any gate on Alice's qubit requires updating the joint state,

which forces a synchronisation barrier every BSM event. For entanglement-heavy topologies (purification chains, repeater networks) this overhead largely cancels the parallelism benefit. For QKD specifically (no long-lived entanglement, no purification) the synchronisation cost is lower — mostly limited to BSM events — so the core-per-node approach is more viable here than for general quantum networking. The practical ceiling is core count: most HPC jobs don't give you exclusive access to thousands of cores simultaneously, so you're typically mapping several simulated nodes per physical core and losing the clean 1:1 analogy.

Slide 10 — Possible Directions

Six strategies. The framing that matters: **what cost does each one actually attack?**

Strategy	Attacks	Inherited from classical?
PDES	Wall-clock	Yes (with caveat)
Fluid/flow	Event count	Yes
State compression	Per-event cost	No — quantum-native
Surrogates	Both	Yes (MimicNet etc.)
Engine optimisation	Constant overhead	Yes
Adaptive sampling	Wasted MC budget	No — quantum-native

PDES caveat (anticipate this question): Chandy-Misra and Time Warp both assume logical processes can be partitioned at a boundary with self-contained state. A Bell pair straddling a partition boundary has no well-defined local state on either side — conservative lookahead has nothing to bound, optimistic rollback would undo a shared density matrix. Only embarrassingly-parallel MC (independent seeds) ports cleanly. This is a genuine open research question for the field.

The core-per-node framing is a natural way to picture PDES for quantum networks: assign one CPU core (or HPC node) per simulated network node, and use inter-core messages to model inter-node traffic. This is cleaner than it sounds for QKD specifically, because QKD has no long-lived shared entanglement — the only synchronisation barrier is the BSM coincidence event, which is localised in time. For general quantum networking (purification, entanglement swapping, quantum memories) the synchronisation cost is much higher because the joint state is non-local and must be locked across cores for every gate. Memory bandwidth, not clock speed, is the per-core bottleneck — density matrices are large and cache-thrashing dominates before the ALU does.

State compression is the one row with no classical twin. Classical packet state doesn't grow exponentially with interaction history. Hilbert space does. Stabiliser/Clifford restriction, MPS, Hilbert truncation — all exact within a restricted class. Good rhetorical point: the

quantum simulation problem is not just a harder version of the classical problem; it has a qualitatively different bottleneck.

Fluid/flow: quantum analogue is replacing per-attempt heralded entanglement simulation with analytic waiting-time distributions. Shchukin, Schmidt & van Loock (PRA 100, 2019); Brand, Coopmans & Wehner. Generating-function work on swap-ASAP chains gets exact fidelity moments up to 25 segments without any MC — exact statement of the analogy.

Engine optimisation: zero fidelity cost, natural first thing to try. Cache repeated repeater segments in pydynaa. Currently underused.

Surrogate modelling: see slide 12 for qualification of the speedup claim. Don't lead with the number here.

Adaptive sampling: see slide 11. 30% saving for BB84; blocked for MDI by spawn overhead and η_d^2 variance.

Slide 11 — Adaptive Sampling

Hypothesis: stop MC early when $\sigma/\mu < 10\%$ (batches of 100); use full budget near QBER cutoff.

Run saving vs wall-clock saving distinction: BB84 runs 90% fewer MC samples (stops at 100 vs 1000). But wall-clock saving is only 30% because each batch still pays the spawn overhead (~2-3s). Spawn overhead is fixed per batch, not per run. Early stopping at 100 runs = 1 spawn call; fixed-1000 = 10 spawn calls. The saving comes from reducing spawn count, not from fewer runs per se.

MDI — why it never stops early: η_d^2 coincidence means even in the stable regime (short distances, high rates), the estimator variance is structurally high. σ/μ never drops below 10% threshold. Adaptive sampling never triggers. Run saving = 0%, wall-clock $\approx 0\%$. The low relative error in the MDI figure is not despite adaptive sampling — it's because adaptive sampling never activates, so MDI just runs the full 1000 every time (identical to baseline).

Third MDI bullet — "requires in-loop sampling": current implementation checks convergence between externally spawned batches. To fix MDI you'd need the σ/μ check inside the simulation process itself — sample one run at a time, halt when criterion met, no respawn between checks. Avoids spawn cost entirely but requires refactoring the MC loop away from multiprocessing at the outer level.

Scale dependence: at small budgets, spawn overhead dominates and adaptive sampling can hurt (adds complexity for no gain). The 30% saving only appears at large fixed budgets (1000 runs) where 10 spawns vs 1 is significant.

Slide 12 — Surrogate Modelling

The number ($3 \times 10^6 \times$) needs qualification before quoting:

1. **Denominator is expensive by design.** 28s/point used 500 MC runs — a high-quality budget. Cheaper baseline (50 runs) shrinks the ratio by $10 \times$.
2. **Numerator ignores training cost.** 256 training points $\times$ 28s $\approx$ 2 hours front-loaded. The 88ms inference only wins if you evaluate the dense grid many times. One-shot use: no advantage.
3. **Grid density amplifies the ratio.** 10,000 evaluation points vs 256 training points — the ratio scales with grid size. Dense grids look better by construction.
4. **Not tested at network scale.** All results are P2P BB84. Network simulation per point is orders of magnitude slower; GP training cost scales too.

True position vs other strategies: most promising asymptotic direction, least battle-tested. Engine optimisation and adaptive sampling deployable now with zero fidelity cost. Surrogates require upfront investment and only pay off for dense repeated parameter sweeps.

GP advantages beyond speedup: unlike other surrogate types, GP gives you an uncertainty estimate at every predicted point at no extra cost — the same computation that gives you the key rate estimate also tells you how confident the model is. That uncertainty map (right panel) shows where the surrogate needs more training data, so you can direct simulation budget exactly where it matters rather than sampling blindly. Smooth interpolation over arbitrary parameter combinations. Train once, re-evaluate instantly for any optimisation.

Prieler et al. (2024) use SVR/RF instead of GP — different surrogate family, no uncertainty quantification. Noted as a comparison point.

If asked what a Gaussian Process actually is:

Instead of fitting a fixed-form function (polynomial, neural net), a GP defines a probability distribution *over functions*. You're not asking "what is the key rate here?" — you're asking "what functions are consistent with my training data, and what do they predict here?"

Two ingredients:

- **Mean function** — often zero; let the data speak
- **Kernel/covariance function** — encodes how correlated outputs should be at nearby inputs. RBF kernel: — nearby points in parameter space $\rightarrow$ similar key rates.

At a new test point, the GP computes posterior mean (kernel-weighted interpolation of training outputs) and posterior variance (zero at training points, grows where data is sparse) — both from the same matrix solve. No second pass needed for uncertainty.

Practically: you run 256 NetSquid simulations to train, then evaluate the GP instantly at 10,000 parameter combinations. The variance map tells you which regions the surrogate is guessing vs. confident about — so you can direct new simulation budget there rather than sampling blindly.

Cost: GP training scales as in training points (matrix inversion). At 256 points fine; at 10,000+ it becomes the new bottleneck.

Slide 13 — Further Limitations

Statistical:

- Failed MC runs excluded before aggregation — skews lower error bars near QBER cutoff (excluded runs are precisely the noisy ones).
- Asymptotic key rate only — no finite-key or composable security corrections.
- Block size $n=1024$ is small; finite-key penalty not quantified.

Protocol:

- Ideal sifting — no error correction or privacy amplification overhead simulated.
- Timing jitter not modelled.
- Ideal single-photon source assumed (no WCP/decoy extension in main results — that's the backup slide).

Network:

- No multi-user scheduling — all pairs simulated independently with no contention model.
- No WDM — single pair per relay at a time.
- No quantum memory model — limits repeater extension to full-generation protocols only.

Validation:

- Benchmarked against PLOB bound (Pirandola et al., 2017) — not validated against published experimental key rate data.
 - Relay placement assumed k-means optimal — not benchmarked against ILP or brute force.
-

Slide 14 — Lessons and an Open Question

Four lessons from running NetSquid at this scale:

1. DES is powerful at P2P and small networks — arbitrary protocol logic, noise composition, no rederivation.
2. Computationally intractable at metropolitan scale on a single machine.
3. The single-threaded event loop is a **fundamental constraint**, not an implementation detail — intrinsic to DES.
4. Quantum memories compound cost — the problem gets harder as the physics gets richer.

Open question: optimise existing DES via classical techniques (Chandy-Misra PDES, surrogates, adaptive sampling) — or does quantum causal structure demand bespoke simulation frameworks entirely?

The Chandy-Misra reference is deliberate: 40+ years old, proven for classical networks, but quantum entanglement breaks the lookahead assumption. That's the precise point of tension.

Slide 15 — Summary

Three bullet takeaways:

1. NetSquid is powerful for arbitrary noise composition and protocol logic — verifiable against PLOB.
 2. BB84 vs MDI: η_d^2 dependence emerges naturally; MDI infrastructure scales $O(N)$ vs $O(N^2)$.
 3. Binding constraint is computational cost at large N ; distributed compute nearest-term fix; deeper question is whether DES is the right abstraction at scale.
-

Anticipated questions (general)

"What's the right simulator for large quantum networks then?"

Open question — no consensus. QuISP, SimulaQron, SeQUeNCe each make different trade-offs. NetSquid's event-loop granularity is its strength and its bottleneck simultaneously.

"Have you validated against experiment?"

Validated against PLOB bound and qualitative η_d^2 scaling. Not yet against published experimental key rate data — acknowledged limitation.

"Could you use Time Warp if you restrict to separable states?"

Possibly — for subprotocols that don't generate entanglement across partition boundaries. Open research direction. Good closing point for the open question slide.

"Why k-means for relay placement?"

Standard unsupervised clustering; centroid placement confirmed optimal for single-cluster case by grid sweep (relay_placement.py exp 1). Two-cluster case: centroid vs boundary placement compared. K-means is a reasonable heuristic; ILP would give global optimum but is computationally expensive for large N.

"Why doesn't NetSquid implement the fixes I suggest?"

Most recent change pushed to live was in 2023, seems to be out of active development. Limitations due to qubit processing may have exacerbated authors' desire to pursue further. (non-answer?)

Backup Notes — Why GPU Doesn't Help

The bottleneck is control flow and process management, not floating-point throughput.

- **Sequential event loop:** causally ordered within each run — cannot be batched on GPU.
- **Spawn overhead:** OS process management; GPU arithmetic irrelevant.
- **$O(N^2)$ pair count:** GPU doesn't reduce pair count.
- **Matrix sizes:** BB84/MDI never exceed 4×4 density matrices. GPU overhead dominates for operations this small. CPU is faster.
- **NetSquid has no GPU backend** — Python + C extensions, CPU-only.

GPU would help if the bottleneck were large matrix operations (e.g., many-qubit state evolution). It doesn't help here.

If asked about reformulating as a linear algebra problem to enable GPU: viable for BB84/MDI specifically. Express the network as a weight matrix where encodes transmission efficiency (loss, , etc.), then key rates for all pairs = elementwise matrix operations — one GPU kernel launch. Parameter sweep = update , recompute. Exact, not stochastic.

Breaks down when protocols have conditional logic on stochastic outcomes (heralded entanglement, purification retries, swap-and-wait). CUDA's warp model requires all 32 threads in a warp to execute the same instruction — stochastic branching causes warp divergence and serialises execution. Entanglement itself isn't the problem; the conditional retry logic around heralding is. Deterministic fixed operations are fine on GPU.